

Materials:

- ① RPi Pico
- ② Jumper wires (x3)
- ③ Joystick (5pin)

For Servo Motor

GND → GND
+5V → 5 or 3.3 volts
VRX → Pin 27
VRY → Pin 26
SW → Pin 22

Inner workings

- Imagine two potentiometers
 - ↳ ① horizontal (0 to 100%)
 - ↳ ② vertical (0 to 100%)
- As you turn the joystick, the output voltage varies
- We convert this analog (continuous) signal to a digital one!

Remember → A.D.C. → Analog to Digital Converter

↳ We can represent 16-bits, so a total of
 $2^{16} = 65,536$

$0 \leq \text{Value} \leq 65,535$

- • ① horizontal ($0 \leq \text{Value} \leq 65,535$)

0% → 100%

- ② vertical ($0 \leq \text{Value} \leq 65,535$)

0% → 100%

Example ③:

```

1 import machine
2 import time
3
4 # First, declare the analog joystick pins (x = horizontal and y = vertical)
5 x_Joystick = machine.ADC(machine.Pin(27))
6 y_Joystick = machine.ADC(machine.Pin(26))
7
8 while True:
9
10     # Let's read the input values (16 bit - 0 to 65,535)
11     x_Value = x_Joystick.read_u16()
12     y_Value = y_Joystick.read_u16()
13
14     # Let's print this to the shell so we can monitor the range of values
15     print(x_Value, y_Value)
16     time.sleep(0.5)

```

Note: Linear Eq help Filter Sensor Data

- we can monitor numbers, but what do they mean?
- we know we want $0 \leq \text{value} \leq 65,535$
 - ↳ however $\rightarrow$ the low # is like 272 or 288
 - ↳ In the real world we have noise
 - ↳ we know this is a semi-linear joystick, so we can just apply our linear equation again!

Point - Slope Formula - Represent specific point on Line

↳ We have to do an equation for each axis!

$$y - y_1 = m(x - x_1) \quad ; \quad x_{11} = 272; x_{12} = 65,535$$

$$y_{11} = 0 \quad ; \quad y_{12} = 180$$

Let's think in degrees!

$$\text{Slope (m)} = \frac{y_2 - y_1}{x_2 - x_1} = \frac{180 - 0}{65,535 - 272} = 0.00276$$

$$y - 0 = 0.00276(x - 272) \rightarrow y = 0.00276x - 0.750$$

$$\text{Equation (Horizontal)} \rightarrow y = 0.00276x - 0.750$$

Example ④:

```

1 import machine
2 import time
3
4 # First, declare the analog joystick pins (x = horizontal)
5 x_Joystick = machine.ADC(machine.Pin(27))
6
7 while True:
8
9     # Let's read the input value (16 bit - 0 to 65,535)
10    x_Value = x_Joystick.read_u16()
11
12    x_Calibrate = int(0.00276*(x_Value) - 0.750)
13
14    # Let's print this to the shell so we can monitor the range of values
15    print(x_Calibrate)
16    time.sleep(0.2)

```

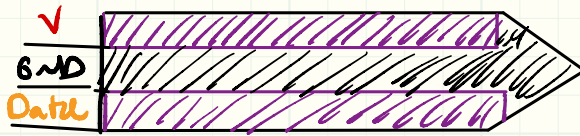
- Now, can we put this together with a Servo
 ↳ Let's control a servo with the joystick x-axis!

Example ⑤:

```

1 # This script will calibrate a noisy joystick, determine the user defined angle, and control a servo motor (one axis)
2
3 import machine
4 import time
5
6 # First, declare the analog joystick pins (x = horizontal)
7 x_Joystick = machine.ADC(machine.Pin(27))
8
9 # Next, set up the servo pin and PWM characteristics
10
11 servo = machine.PWM(machine.Pin(15))
12 servo.freq(50)
13
14 # Always good practice to initialize the servo (set to origin point)!
15 servo.duty_u16(1638)
16
17 while True:
18
19     # Let's read the input value (16 bit - 0 to 65,535)
20    x_Value = x_Joystick.read_u16()
21
22    # Next, let's calibrate the reading from the joystick to a linear function
23    x_Calibrate = int(0.00276*(x_Value) - 0.750)
24
25    # We can see that this "x_Calibrate" actually represents our servo angle (this is why we said 0 to 180)!
26    angle = x_Calibrate
27
28    # Next, remember the linear equation for the servo motor from scripts 1 and 2!
29    duty_value = (6553/180)*angle + 1638
30
31    # Now, assign the input from the joystick (the angle) to the servo motor!
32    servo.duty_u16(int(duty_value))
33
34    print(int(angle))
35
36    # Next,
37    time.sleep(0.2)
38

```



$$C = \frac{\epsilon A}{d}$$

- Two plates have an electric field between them
- When moisture is between the plates → Dielectric Constant ϵ increases
- Higher moisture ↑ Lower Analog Signal
Lower moisture ↓ higher Analog signal

Example ①:

```

1 # This script will read the soil moisture level from a capacitive soil moisture sensor.
2 import machine
3 import time
4
5 # Set up the soil moisture sensor as an analog input on an ADC pin.
6 soil_sensor = machine.ADC(machine.Pin(27))
7
8 # Execute the following commands continually
9 while True:
10
11     # Read the moisture level as an input from the soil sensor.
12     soil_moisture = soil_sensor.read_u16()
13
14     # Print the soil moisture level to the **shell** for the user to read.
15     print('The Soil Moisture Level Is: ', soil_moisture)
16
17     time.sleep(0.2)

```

Example (2):

```

1 # This script will read the soil moisture level from a sensor, fit the reading to a linear equation, and control an LED.
2 import machine
3 import time
4
5 # Set up the soil moisture sensor as an analog input on an ADC pin.
6 soil_sensor = machine.ADC(machine.Pin(27))
7
8 # Set the on-board LED as an output (indicator).
9 led = machine.Pin("LED", machine.Pin.OUT)
10
11 while True:
12
13     # Read the raw sensor value.
14     sensor_reading = soil_sensor.read_u16()
15
16     # We know the minimum value (full squeeze) is 32,000 and the maximum value (no squeeze) is 57,000 (rough estimate).
17     # Using this, we will create a linear equation to represent this data ; full-squeeze is 100 and no-squeeze is 0.
18     x_1 = 57300
19     x_2 = 31000
20     y_1 = 0
21     y_2 = 100
22
23      $y - y_1 = m(x - x_1)$ 
24     m = (y_2 - y_1) / (x_2 - x_1)
25     soil_moisture_level = int(m * (sensor_reading - x_1) + y_1)
26
27     # This sensor isn't meant to be a pressure sensor because it capacitively couples to the body (which is hard to calibrate).
28     # There is a chance we would read above 100 or below 0, so we will clamp our reading to make sure we have "good values".
29     soil_moisture_level_clamped = [int(max(0, min(100, soil_moisture_level)))]
30
31     print('The Soil Moisture Level is: ', soil_moisture_level_clamped)
32
33     # Trigger the LED if and only if the soil moisture level is above a certain value
34     if soil_moisture_level_clamped >= 85:
35         led.on()
36
37     else:
38         led.off()
39
40     time.sleep(0.2)

```

If the sensor reads below zero → give zero
above 100 → give 100
otherwise → real value (int)

Example (3):

```

1 # This script will read the "squeeze" level from a capacitive sensor, fit the reading to a 0-180 degree angle, and control a servo motor.
2 import machine
3 import time
4
5 # Set up the soil moisture sensor as an analog input on an ADC pin.
6 soil_sensor = machine.ADC(machine.Pin(27))
7
8 # Set up the servo pin and PWM characteristics.
9 servo = machine.PWM(machine.Pin(15))
10 servo.freq(50)
11
12 # Always good practice to initialize the servo (set to 0 degrees)!
13 servo.duty_u16(1638)
14
15 while True:
16
17     # Read the raw sensor value.
18     sensor_reading = soil_sensor.read_u16()
19
20     # We know the minimum value (full squeeze) is 32,000 and the maximum value (no squeeze) is 57,000 (rough estimate).
21     # Using this, we will create a linear equation to represent this data ; full-squeeze is 180 and no-squeeze is 0.
22     # Here, we are using 180 as the max value, because a servo motor can move a maximum of 180 degrees.
23     x_1 = 57300 # "No squeeze" reading (maps to 0 degrees)
24     x_2 = 31000 # "Full squeeze" reading (maps to 180 degrees)
25     y_1 = 0
26     y_2 = 180
27
28     m = (y_2 - y_1) / (x_2 - x_1) ①
29
30     # Calculate the angle based on the squeeze.
31     servo_angle_raw = m * (sensor_reading - x_1) + y_1
32
33     # This sensor isn't meant to be a pressure sensor because it capacitively couples to the body (which is hard to calibrate).
34     # There is a chance we would read above 180 or below 0, so we will clamp our reading to make sure we have "good values".
35     servo_angle_clamped = int(max(0, min(180, servo_angle_raw)))
36
37     print('Servo Angle is: ', servo_angle_clamped)
38
39     # Next, remember the linear equation for the servo motor from last class (1638 = 0 degrees and 8191 = 180 degrees)!
40     duty_value = (6553 / 180) * servo_angle_clamped + 1638 ②
41
42     # Now, assign the input from the squeeze sensor (the angle) to the servo motor!
43     servo.duty_u16(int(duty_value))
44
45     time.sleep(0.2)

```

Two Linear Equations!